Dynasties Phase One Framework v4

The Agentic Operating System for Global Trade

Prepared for: Dynasties.ai Founding Team

Date: March 13, 2026

Author: Manus AI

Classification: Internal — Execution Blueprint

Executive Summary

Dynasties is not a better freight forwarding system. It is the first agentic operating system for global trade — a platform where AI agents perform operational work, orchestrate workflows, and compound intelligence from every shipment, while humans govern outcomes rather than execute tasks. This document is the buildable blueprint for Phase One.

The strategic positioning draws from two frameworks. **Blue Ocean Strategy** [1] teaches that the most valuable companies do not win by competing harder in existing markets — they create new markets where competition is irrelevant. Every incumbent logistics platform (Magaya, CargoWise, Descartes, e2open, Flexport) competes in the same red ocean: workflow databases where humans perform the work. They compete on features, integrations, and price. Dynasties exits that ocean entirely by redefining what a logistics platform does. It does not store data that humans enter. It performs the operational work itself.

Peter Thiel's Zero to One [2] teaches that transformative companies create something genuinely new — a 0 to 1 leap — rather than incremental improvements (1 to n). The 0 to 1 insight for Dynasties is this: the freight forwarding industry does not have a technology problem. It has a labor problem disguised as a technology problem. Every TMS on the market is a tool that makes human labor slightly more efficient. Dynasties eliminates the labor. When an email arrives containing a booking request, the system does not help an operator enter the data faster — it reads the email, extracts the data, classifies the commodity, screens for sanctions compliance, drafts the shipment record, calculates the charges, generates the shipping documents, and queues everything for a single human approval. The operator's job shifts from performing work to governing outcomes.

The product experience is captured in one sentence: **"Forward your emails, documents, and legacy exports to Dynasties, and the platform automatically assembles a working operational system around your business."** This is not a feature. It is the entire product

philosophy. Dynasties transforms unstructured operational inputs — emails, PDFs, spreadsheets, XML files, scanned documents — into structured operational workflows. The system learns from every interaction, building a persistent Company Memory Graph that encodes each client's customers, carriers, trade lanes, commodities, pricing patterns, approval policies, and operational preferences. Over time, the platform becomes increasingly autonomous — not because it was programmed to handle every case, but because it learned how each company operates.

The architecture enforces a cardinal rule: **LLMs reason, but they never execute.** Agents produce structured outputs — classifications, recommendations, extracted data, generated text. Deterministic services execute system actions — creating database records, generating documents, processing payments. A seven-layer execution stack ensures that every AI output passes through validation, business rules, and human approval before it touches the system of record. This is what makes the platform production-grade rather than a prototype.

Phase One ships in 30 days. It delivers the universal intake layer (email, documents, CSV, XML, legacy exports), the Company Memory Graph, the Agentic Workbench, shipment creation, compliance screening, commodity classification, document generation, charge calculation, invoicing, cross-platform data import, and a natural language operational interface. Insurance integrations, advanced analytics, and deep carrier API integrations are deferred to Weeks 5-8.

---

1. Strategic Positioning

The Red Ocean

Global trade logistics software is a $15 billion market [3] dominated by platforms that all do the same thing: provide a database with forms where humans enter shipment data, generate documents from templates, and track cargo through manual milestone updates. The competitive landscape is crowded and undifferentiated.

Platform	What It Actually Is	Annual Revenue	Founded
CargoWise (WiseTech Global)	Workflow database with deep customs integrations	~$700M [4]	2000
Descartes Systems	Logistics network + compliance database	~$550M [5]	1981
Magaya	Windows desktop TMS for small-to-mid	~$50-100M (est.)	2001

	forwarders		
e2open	Supply chain planning + visibility platform	~$500M [6]	2000
Flexport	Digital forwarder with proprietary TMS	~$1B+	2013

These companies compete on the same dimensions: number of features, number of integrations, geographic coverage, and price per user. They are all workflow databases. They all require humans to perform the operational work. They all treat intelligence as an optional add-on rather than the core product. This is the red ocean.

The Blue Ocean

Dynasties creates a new market space by competing on dimensions that incumbents cannot match — because matching them would require rebuilding their products from scratch.

Red Ocean Dimension (Incumbents)	Blue Ocean Dimension (Dynasties)
Humans enter data into forms	Agents extract data from emails and documents
Static workflow templates	Adaptive workflows that learn from operator behavior
Intelligence as an add-on module	Intelligence as the core operating layer
Per-user pricing ($300-$450/user/month)	Per-shipment pricing (cost scales with value delivered)
Migration requires months of IT work	Migration requires forwarding emails
System stores what humans type	System performs the work humans used to do
Dashboard shows historical data	Workbench surfaces decisions that need governing
Isolated per-company data	Cross-client intelligence network (anonymized)

The Blue Ocean framework identifies four actions that create new market space: **Eliminate** (what the industry takes for granted that should be removed), **Reduce** (what should be

reduced below the industry standard), **Raise** (what should be raised above the industry standard), and **Create** (what should be created that the industry has never offered) [1].

Eliminate: Manual data entry. Form-based interfaces. Per-user licensing. Multi-month implementation projects. Dedicated IT staff for system maintenance.

Reduce: Operator time per shipment (from 15-30 minutes to under 60 seconds). Number of screens to complete a workflow (from 8-12 to 1 approval view). Training time for new operators (from weeks to hours). Dependence on any single operator's institutional knowledge.

Raise: Speed of first value delivery (from months to minutes). Data quality (AI extraction is more consistent than human data entry). Compliance coverage (100% of shipments screened, not the 30-40% that get manual review). Operational visibility (every agent action is logged, reasoned, and auditable).

Create: The Company Memory Graph — a persistent intelligence layer that learns how each company operates. The Agentic Workbench — an interface where operators govern AI-driven operations rather than performing manual tasks. The Universal Intake Layer — a single ingestion point that transforms any operational input into structured workflows. Learning Loops — a system that gets smarter with every operator correction.

The Zero-to-One Advantage

Peter Thiel's framework asks: "What important truth do very few people agree with you on?" [2] The Dynasties answer: **freight forwarding operations can be performed by AI agents, not just assisted by software tools.** Every incumbent believes that logistics software should help humans work faster. Dynasties believes that logistics software should do the work, with humans governing the outcomes.

This is a 0 to 1 leap because it changes the unit economics of freight forwarding. Today, a forwarder's cost structure is dominated by labor — operators who enter data, generate documents, check compliance, calculate charges, and send invoices. Dynasties collapses that labor cost by 80-90%, enabling a forwarder to handle 10x the shipment volume with the same team. This is not an incremental improvement. It is a structural transformation of the business model.

The technological moat compounds over time. Every shipment processed through Dynasties trains the Company Memory Graph. Every operator correction improves the agents. Every new client adds data to the cross-client intelligence network. Incumbents cannot replicate this advantage by adding AI features to their existing platforms — their architectures are built around human data entry, and retrofitting agent-native intelligence into a 15-year-old Windows application or a monolithic Java codebase is a multi-year rewrite that would alienate their existing customer base.

2. Product Philosophy

The Core Promise

> "Forward your emails, documents, and legacy exports to Dynasties, and the platform automatically assembles a working operational system around your business."

This sentence is the entire product. Every architectural decision, every feature priority, and every design choice must serve this promise. If a feature does not contribute to transforming unstructured operational inputs into structured operational workflows, it does not belong in Phase One.

The Universal Intake Layer

Dynasties must ingest operational data from anywhere, in any format, at any time. The intake layer is not a feature — it is the front door of the platform. Everything begins here.

Input Type	How It Enters	What the System Does
Email	Operator forwards or BCCs to `ops@dynasties.ai`	Parses MIME content, extracts attachments, identifies document types, detects shipment intent, resolves parties, extracts commodity data, drafts shipment record, queues for approval
PDF attachments	Attached to emails or uploaded directly	OCR + Claude Sonnet 4.6 extraction. Handles BOLs, commercial invoices, packing lists, certificates of origin, customs declarations, arrival notices
DOCX / XLSX attachments	Attached to emails or uploaded directly	Structured parsing for spreadsheets, text extraction for Word documents
CSV / Excel imports	Uploaded via web interface or drag-and-drop	Column mapping (AI-assisted for unknown formats, pre-built for Magaya/CargoWise), batch validation, enrichment pipeline
XML / EDI files	Uploaded or received via integration endpoint	Schema-aware parsing using published XSD definitions

		(Magaya, INTTRA, EDIFACT)
Legacy platform exports	Uploaded via Migration Studio	Pre-built parsers for Magaya, CargoWise, Descartes. AI-assisted mapping for unknown formats
Document repository uploads	Bulk upload via web interface	Batch OCR, document classification, entity extraction, chronological ordering
API ingestion	REST API endpoint	Structured JSON input for programmatic integrations with external systems
Manual entry	Web interface or natural language	Traditional form entry or natural language: "Ocean FCL, 2x40HC, auto parts from Busan to Houston"

Email as a first-class operational interface is the critical design decision. Most freight forwarding work begins with an email. A customer sends a booking request. A carrier sends a rate quote. A customs broker sends a filing confirmation. An overseas agent sends an arrival notice. In traditional systems, the operator reads the email, then opens the TMS, then manually re-types the information from the email into the system. Dynasties eliminates that re-typing entirely.

From a single email thread, the system performs the following sequence automatically: detect that the email contains shipment intent (booking request, quote request, shipping instruction), extract all attached documents and classify them by type, parse each document for structured data (parties, addresses, commodity descriptions, quantities, weights, dimensions, values, dates, reference numbers), resolve extracted party names against the Company Memory Graph (matching "Samsung Electronics Co., Ltd." to the existing customer record for Samsung), identify commodities and suggest HS codes, screen all parties against sanctions and restricted party lists, draft a complete shipment record with all extracted fields populated, draft charges based on the company's rate tables and historical pricing patterns, generate operational documents (BOL, manifest, arrival notice) from the drafted shipment, queue the entire package for operator approval in the Agentic Workbench, and record the interaction in the Company Memory Graph to improve future processing of similar emails.

The operator's experience: they see a notification in the Workbench — "New shipment drafted from email. Origin: Shenzhen. Destination: Miami. Commodity: Consumer

Electronics. HS Code: 8471.30 (confidence: 94%). All parties cleared. Estimated charges: $4,200. Documents generated: BOL, Manifest, Commercial Invoice." They review, adjust if needed, and click "Approve." Total time: under 60 seconds.

---

3. Agent Architecture

The Cardinal Rule

LLMs reason. Services execute. The system of record is never written to by an LLM.

This rule is absolute. It is the difference between a production system and a demo. Every AI-agent failure in enterprise software traces to the same root cause: a language model was given authority to modify state directly, and it hallucinated a carrier name, invented a charge amount, or misclassified a commodity — and the bad data went straight into the database because nothing sat between the model's output and the system action.

Dynasties enforces a seven-layer execution stack. Data flows through these layers in sequence. No layer can be skipped. No shortcut exists.

The Seven-Layer Execution Stack

Layer 1 — Deterministic Workflow Engine. Built on LangGraph [7], this layer defines the exact sequence of steps for every business process. It is pure orchestration — no AI logic, no probabilistic behavior. Given the same inputs, it produces the same step sequence every time. It manages state transitions, checkpointing (so workflows can be paused and resumed), retry logic (so transient failures do not kill a workflow), and escalation paths (so stuck workflows surface to operators). Every workflow is a directed graph with defined entry points, decision nodes, parallel branches, and terminal states.

Layer 2 — Rules Engine. Before any AI agent is invoked, the rules engine evaluates whether AI is even needed. Many freight operations are deterministic: if the commodity is "lithium-ion batteries" and the destination is any country, the compliance screening must include IATA DGR Class 9 verification — this is a rule, not a judgment call. The rules engine handles these cases without invoking an LLM, saving cost and latency. Rules are defined as structured YAML conditions maintained by the operations team. The rules engine also enforces hard constraints that agents cannot override: maximum charge amounts, mandatory compliance checks, required approval thresholds, and regulatory filing requirements.

Layer 3 — Model Router. When the rules engine determines that AI reasoning is required, the model router selects the appropriate LLM. The router is a deterministic function — it evaluates task metadata (type, complexity, context size, cost budget) and routes to the correct model. Full architecture detailed in Section 7.

Layer 4 — Agent Reasoning Layer. This is where LLMs operate. Each specialist agent receives a structured JSON input, processes it through the selected model with a task-specific versioned prompt, and returns a structured JSON output with confidence scores. Agents never see the database. They never execute system actions. They produce recommendations, classifications, extractions, and generations — always as structured data.

Agent	Domain	Model	Input → Output
Intake Agent	Email parsing, document classification, data extraction	Claude Sonnet 4.6	Raw email + attachments → Structured shipment draft (JSON) with field-level confidence
Classification Agent	HS code suggestion with tariff schedule RAG	GPT-5.4	Commodity description + origin + destination → HS code candidates with confidence and reasoning
Compliance Agent	Sanctions screening, export control, denied party checks	GPT-5 Mini	Entity names + addresses + commodity-destination pairs → Pass/fail per entity with match details
Carrier Agent	Carrier selection, benchmarking, rate comparison	GPT-5.4	Route + commodity + timeline + dimensions → Ranked carrier list with cost, transit, reliability
Document Agent	BOL, AWB, manifest, arrival notice, invoice generation	Claude Sonnet 4.6	Complete shipment record → Document content ready for template rendering
Pricing Agent	Charge calculation, margin analysis, anomaly detection	GPT-5 Mini	Shipment details + rate tables + historical patterns → Itemized charges with margin analysis

Entity Agent	Party resolution, deduplication, address normalization	GPT-5 Mini	Raw party names/addresses → Matched or new entity records with confidence
Migration Agent	Legacy data parsing, field mapping, validation	Claude Sonnet 4.6	Raw CSV/XML rows + source system ID → Mapped records conforming to Dynasties schema
Memory Agent	Pattern recognition, preference learning, SOP extraction	GPT-5.4	Operator corrections + historical decisions → Updated Company Memory Graph entries

Layer 5 — Tool Execution Layer. Agent outputs are structured recommendations. The tool execution layer converts those recommendations into system actions — but only after passing through the approval layer (Layer 6). Tools are deterministic functions with input validation, error handling, and rollback capability: `create_shipment(data)`, `generate_document(template, data)`, `calculate_charges(shipment_id, rate_table)`, `send_email(recipient, content)`, `create_invoice(charges)`, `update_entity(entity_id, data)`. Tools write to the system of record. Agents do not.

Layer 6 — Human Approval Checkpoints. Every workflow has defined approval gates where execution pauses and waits for operator confirmation. The operator sees the agent's recommendation alongside the confidence score and reasoning. They can approve (proceed to execution), modify (edit the recommendation and proceed), or reject (cancel and optionally provide feedback). Approval gates are configurable per client — a new client might require approval on every action, while a mature client might auto-approve actions above 95% confidence. Every approval or correction is recorded and fed back to the Memory Agent (Layer 4) to improve future recommendations.

Layer 7 — System-of-Record Transaction Layer. All writes to the database go through a transactional service that enforces ACID properties, maintains audit logs, and ensures referential integrity. Every record includes provenance metadata: which agent produced the recommendation, which model was used, the confidence score, whether a human approved or modified it, and the timestamp of every state change. This audit trail is not optional — it is what regulators, compliance officers, and customers require.

End-to-End Flow: Email to Live Shipment

An operator forwards an email containing a booking request and commercial invoice to `ops@dynasties.ai`.

The **Workflow Engine** (Layer 1) receives the email event and initiates the "New Shipment from Email" workflow, defining the step sequence: parse → extract → resolve entities → classify → screen compliance → draft shipment → draft charges → generate documents → queue approval.

The **Rules Engine** (Layer 2) evaluates: Is this sender a known customer? (Pre-populate customer record from Memory Graph.) Does the commodity match any cached classifications? (Skip Classification Agent, use cached HS code.) Are all parties already in the entity database? (Skip Entity Agent for known parties.)

The **Model Router** (Layer 3) routes document parsing to Claude Sonnet 4.6, classification to GPT-5.4 with tariff RAG, and compliance screening to GPT-5 Mini.

The **Agent Reasoning Layer** (Layer 4) processes each task in parallel where possible and returns structured JSON: extracted shipment fields with confidence scores, HS code suggestions with reasoning, compliance results with match details, carrier recommendations with cost analysis, and generated document content.

The **Human Approval Checkpoint** (Layer 6) presents the unified result in the Agentic Workbench. The operator reviews the complete package and clicks "Approve."

The **Tool Execution Layer** (Layer 5) executes: creates the shipment record, generates PDF documents, calculates and records charges, schedules the invoice.

The **System-of-Record Transaction Layer** (Layer 7) commits all writes atomically, records the audit trail, and emits events that trigger downstream workflows.

The **Memory Agent** records the interaction: this customer, this commodity, this route, these charges, this carrier — strengthening the Company Memory Graph for future shipments.

---

4. Company Intelligence Graph

What It Is

The Company Memory Graph is a persistent, evolving knowledge structure that represents everything Dynasties has learned about a company's operations. It is not a static database. It is a living intelligence layer that grows with every shipment, every email, every operator correction, and every document processed.

Traditional TMS platforms store records — shipments, invoices, entities. The Company Memory Graph stores **relationships, patterns, preferences, and precedents.** It knows not

just that Company X shipped electronic components from Shenzhen to Miami, but that Company X always uses Maersk for this lane, always requires a certificate of origin, always marks the shipment as "fragile," always applies a 15% markup on freight charges, and always sends the arrival notice to three specific email addresses at the consignee.

What It Learns

The graph continuously ingests intelligence from six sources and organizes it into interconnected knowledge domains.

Knowledge Domain	What the Graph Learns	How It Learns
Customer Patterns	Preferred carriers per lane, standard markup percentages, required document types, billing preferences, communication style, seasonal volume patterns	Every shipment processed, every operator correction, every email thread
Entity Relationships	Which shippers work with which consignees, which agents handle which ports, which customs brokers specialize in which commodities	Entity co-occurrence across shipments, email thread analysis
Commodity Precedents	HS code rulings for specific product descriptions, classification edge cases, duty rate history, restricted commodity flags	Classification Agent outputs, operator corrections, customs ruling databases
Trade Lane Intelligence	Transit times by carrier and route, congestion patterns, seasonal rate fluctuations, port-specific documentation requirements	Historical shipment data, carrier performance tracking, rate quote analysis
Pricing Behavior	Standard charge structures per customer, margin targets per lane, competitive rate benchmarks, charge anomaly thresholds	Invoice history, rate table evolution, operator pricing decisions

Operational SOPs	Company-specific approval workflows, documentation checklists, exception handling procedures, escalation policies	Operator behavior patterns, explicit SOP uploads, correction patterns

How Agents Use It

Every agent in the system queries the Company Memory Graph before producing its output. This is what makes Dynasties fundamentally different from a generic AI tool — the agents do not operate from general knowledge alone. They operate from the specific, accumulated intelligence of each company's operations.

When the Intake Agent processes an email from a known customer, it queries the graph: "What are this customer's standard shipping patterns? What documents do they typically attach? What fields do they usually include or omit?" The agent uses this context to fill gaps in the extracted data — if the customer always ships from the same origin warehouse and the email does not specify an origin, the agent suggests the known default with high confidence.

When the Classification Agent encounters a commodity description, it first checks the graph for precedents: "Has this company shipped this commodity before? What HS code was used? Was it corrected by the operator?" If a precedent exists with high confidence, the agent returns it immediately without invoking the LLM — the rules engine handles this as a deterministic case.

When the Carrier Agent generates recommendations, it queries the graph for preferences: "Does this customer have a preferred carrier for this lane? What transit times have they accepted historically? What is their price sensitivity?" The recommendation is personalized to the company's actual behavior, not generic market data.

The Strategic Moat

The Company Memory Graph is the primary long-term moat of the platform. It creates three compounding advantages that strengthen over time and cannot be replicated by competitors.

Switching cost inversion. Traditional TMS platforms have low switching costs — the data is tabular and exportable. The Company Memory Graph creates high switching costs because it contains intelligence that cannot be exported as a CSV. A company's operational patterns, carrier preferences, classification precedents, pricing behavior, and approval workflows are encoded in a graph structure that took months of real operations to build. Switching to a competitor means starting from zero intelligence.

Increasing autonomy. As the graph accumulates more data, the rules engine handles more cases deterministically, the agents produce higher-confidence outputs, and the approval thresholds can be raised. A company that has been on Dynasties for 12 months requires significantly less operator intervention than a company on Day 1. The platform gets cheaper to operate and more valuable to the customer simultaneously.

Cross-client intelligence. With appropriate anonymization and consent, patterns learned from one company's operations can benefit all companies on the platform. If 50 companies are shipping the same commodity on the same lane, the platform knows the true market rate, the best-performing carrier, and the most common compliance issues — intelligence that no single company could generate alone. This creates a network effect: each new company on the platform makes the intelligence better for every existing company.

---

5. Operator Workbench Design

Design Philosophy

The Agentic Workbench is not a dashboard. Dashboards display historical data and require humans to decide what to do next. The Workbench surfaces **decisions that need governing** — shipments drafted by agents, compliance flags that need review, charge anomalies that need resolution, documents that need approval. The operator's role shifts from "perform the work" to "govern the outcomes."

The interface design draws from three reference points that represent the best of modern enterprise software:

Notion — for its information density without visual clutter. Content is organized in blocks that can be rearranged, filtered, and viewed in multiple layouts (list, board, table, timeline). The Workbench uses the same principle: shipments, approvals, and alerts are blocks that operators arrange according to their workflow.

Linear — for its speed and keyboard-first interaction model. Every action in Linear is reachable via keyboard shortcut. Status changes happen inline without modal dialogs. The Workbench adopts the same philosophy: approve a shipment with a single keystroke, triage compliance flags without opening a new page, bulk-approve low-risk items in one action.

Stripe Dashboard — for its transparency and developer-grade detail. Stripe shows every API call, every webhook, every state transition. The Workbench shows every agent action, every confidence score, every reasoning chain. Operators can drill into any AI decision and see exactly why the system made that recommendation.

Workbench Layout

The Workbench is organized around **operational queues** — not navigation menus, not feature tabs, not module screens. Each queue represents a category of work that needs operator attention.

Queue	What It Contains	Urgency
Drafts	Shipments assembled by agents from emails, documents, or imports. Ready for operator review and approval. Each draft shows extracted data, confidence scores, and agent reasoning.	Standard — process during normal workflow
Missing Information	Shipments where agents could not extract required fields with sufficient confidence. Highlights exactly which fields need operator input.	Standard — resolve to unblock shipment processing
Compliance Flags	Parties or commodities that triggered sanctions screening alerts, export control restrictions, or denied party matches. Cannot be dismissed — must be explicitly resolved.	High — regulatory requirement
Document Mismatches	Cases where extracted data conflicts across documents (e.g., BOL says 10 cartons but commercial invoice says 12). Agent highlights the discrepancy and suggests resolution.	High — prevents downstream errors
Mapping Conflicts	During data import, fields that could not be automatically mapped to the Dynasties schema. Shows the source field, the attempted mapping, and alternatives.	Standard — resolve during import process
Charge Anomalies	Charges that deviate significantly from historical patterns or rate table	Medium — potential revenue impact

	expectations. Agent flags the anomaly and shows the expected range.	
Approvals	Completed shipment packages (shipment record + documents + charges) ready for final approval before execution. One-click or keyboard shortcut to approve.	Standard — final step before execution

Interaction Principles

Zero-click intelligence. When an operator opens a shipment draft, they see everything the agents have assembled — without clicking into sub-pages or expanding accordions. The shipment details, extracted documents, classification results, compliance screening, carrier recommendation, charge calculation, and generated documents are all visible in a single scrollable view. The operator scans, adjusts if needed, and approves.

Transparent reasoning. Every AI-generated field shows a small indicator: a confidence score (displayed as a colored dot — green for >90%, yellow for 70-90%, red for <70%) and an expandable reasoning panel. Clicking the indicator reveals why the agent made that decision: "HS Code 8471.30 suggested because: commodity description matches 'portable digital automatic data processing machines' in Chapter 84, Note 5(A). Previous shipments from this customer used 8471.30 for similar descriptions (3 precedents). Confidence: 94%."

Correction as training. When an operator modifies an agent's output — changing an HS code, selecting a different carrier, adjusting a charge — the correction is captured and fed to the Memory Agent. The system does not just accept the correction; it learns from it. A dialog appears: "You changed the HS code from 8471.30 to 8471.41. Should this apply to: (a) this shipment only, (b) all future shipments with similar descriptions from this customer, (c) all future shipments with this description from any customer?" The operator's choice becomes a rule in the Company Memory Graph.

Keyboard-first workflow. The most common operator actions are mapped to keyboard shortcuts: `A` to approve, `R` to reject, `E` to edit, `N` to move to next item, `F` to flag for follow-up, `⌘+Enter` to approve and advance. An operator processing a queue of 20 drafted shipments can triage them in minutes without touching the mouse.

Batch operations. For high-volume operations, the Workbench supports batch approval. Operators can filter drafts by confidence threshold (e.g., "show all drafts with >95% confidence"), review the summary, and approve all in one action. This is how a forwarder

processing 100+ shipments per day operates efficiently — they focus their attention on the 10-15 shipments that need human judgment and batch-approve the rest.

---

6. Cross-Platform Migration Studio

The Migration Problem

Every freight forwarder considering a new platform faces the same barrier: their operational history is locked in their current system. Customer records, shipment history, rate tables, document templates, carrier relationships — years of accumulated data that cannot be abandoned. Traditional TMS migrations take 3-6 months, require dedicated IT consultants, and cost $50,000-$200,000. This is why forwarders stay on systems they hate.

Dynasties solves this with the **Migration Studio** — a guided, AI-powered import experience that transforms legacy data into a live operational system in hours, not months.

Supported Source Systems

Source System	Import Method	Pre-Built Parser
Magaya	CSV/Excel export from any list view (Shipments, Bookings, Customers, Carriers, Invoices). XML export for individual records. 243 known fields mapped from public XSD schemas [8].	Yes — field dictionary derived from Magaya's published XML schemas
CargoWise	CSV/Excel data extract. XML via eHub/eAdapter. Standard CargoWise field naming conventions.	Yes — common field mappings pre-built
Descartes	CSV/Excel export from reporting modules. EDI message archives.	Partial — common fields mapped, AI-assisted for custom fields
Generic CSV/Excel	Any spreadsheet with shipment data, customer lists, rate tables, or charge history	AI-assisted — Claude Sonnet 4.6 infers field mappings from column headers and sample data
Document repositories	Bulk upload of PDF/image files (BOLs, invoices, customs filings)	OCR + document classification + entity extraction

Historical email archives	Email export (MBOX, EML, or forwarded batch)	Full email ingestion pipeline — same as live email processing

The Three-Stage Import Pipeline

Stage 1 — Parse. The Migration Agent identifies the file format and parses content into a normalized intermediate representation. For known systems (Magaya, CargoWise), the parser uses pre-built field dictionaries that map source field names to Dynasties equivalents. For unknown formats, Claude Sonnet 4.6 analyzes column headers, sample data, and data patterns to infer mappings. All AI-inferred mappings are presented to the operator for confirmation — the system never assumes a mapping is correct without human validation.

Stage 2 — Validate. Every imported record is validated against the Dynasties data model. Required fields are checked. Data types are enforced. Referential integrity is verified (carrier names matched to existing records or flagged for creation). Duplicates are detected. The validation summary is clear and actionable: "4,231 shipments ready to import. 28 records have warnings (missing consignee address). 3 records failed validation (invalid date format). 0 compliance issues detected."

Stage 3 — Enrich. After validation, imported data passes through the intelligence agents. The Classification Agent reviews commodity descriptions and suggests corrected HS codes. The Compliance Agent screens all parties against current sanctions lists. The Pricing Agent analyzes charge patterns and identifies anomalies. This enrichment transforms a static data migration into an immediate value demonstration — the operator sees intelligence applied to their own historical data the moment it enters the system.

The Onboarding Experience

The onboarding flow delivers first value in under five minutes, following three design principles:

One action to start (Apple). The operator does exactly one thing: forward a single email to `ops@dynasties.ai`. That email triggers the full ingestion pipeline and creates a shipment draft. The operator sees the result before they have finished setting up their account. First value is delivered in 30 seconds — before the operator has committed to anything.

Progressive disclosure (Google). Account creation asks for three things: company name, email address, and password. Rate tables, carrier preferences, document templates, and team members are configured later as the operator discovers the need. The system learns

preferences from usage rather than requiring upfront configuration. The Company Memory Graph builds itself from real operations.

Instant gratification (Stripe). The "Free Intelligence Audit" serves as the conversion moment. Before committing to switch, the operator uploads a CSV export from their current system and receives an AI-generated audit report within minutes — showing misclassifications, compliance gaps, cost overruns, and carrier underperformance in their own data. The audit is free. The intelligence is undeniable. The operator has seen what Dynasties can do with their data before they have spent a dollar.

The Parallel Run Model

Migration does not require the operator to stop using their current system immediately. Dynasties supports a parallel running period where the operator continues using their legacy TMS for active shipments while simultaneously forwarding emails to Dynasties. Both systems process the same data. The operator compares results — what their current system required them to do manually versus what Dynasties did automatically.

The parallel run serves two purposes. First, it builds operator confidence. They see Dynasties handle real shipments correctly before they commit to switching. Second, it builds the Company Memory Graph. Every email forwarded during the parallel period adds intelligence — customer relationships, carrier preferences, commodity patterns, pricing behavior. By the time the operator switches fully, Dynasties already knows their business. The "switch" is not a migration event. It is the recognition that the migration already happened.

---

7. AI Model Routing Architecture

Why Multi-Model Routing

A single-model architecture — using GPT-5.4 for everything — is the simplest approach and the worst for production. It fails on cost (compliance screening does not need a frontier reasoning model), capability (Claude Sonnet 4.6 outperforms GPT-5.4 at document understanding [9], Gemini 2.5 Pro handles 1M token contexts that exceed GPT-5.4's window), and resilience (a single-model architecture has a single point of failure).

The Model Routing Service is a standalone service that all other services call when they need LLM inference. No service calls LLM APIs directly. This centralization provides routing logic, fallback chains, response caching, confidence scoring, and prompt version management in a single auditable layer.

Model Assignment

Task Type	Primary Model	Why This Model	Fallback	Cost per 1M Tokens (In/Out)
Complex reasoning, shipment orchestration, multi-step decisions	GPT-5.4	Best agentic capabilities, strongest tool use [10]	Claude Sonnet 4.6	~$2.50 / ~$10.00
Document parsing, OCR post-processing, communication generation	Claude Sonnet 4.6	Superior document understanding, nuanced text generation [9]	GPT-5.4	~$3.00 / ~$15.00
Large-context analysis (full shipment history, bulk document review)	Gemini 2.5 Pro	1M token context window, cost-effective for large inputs [11]	Claude Sonnet 4.6	~$1.25 / ~$5.00
High-volume operational tasks (compliance checks, charge calculations, entity matching)	GPT-5 Mini	10x cheaper than frontier models, sufficient for pattern-matching tasks [10]	GPT-5.3 Instant	~$0.25 / ~$2.00

Fallback Chains

If the primary model returns an error, times out, or returns a response below the confidence threshold, the router automatically retries with the fallback model. The fallback is transparent to the calling service — it receives a response regardless of which model produced it, with metadata indicating which model was used. Fallback chains are defined per task type and can be updated via configuration without code deployment.

Response Caching

Identical inputs produce identical outputs for many tasks. The classification of "electronic integrated circuits" as HS 8542.31 does not change between calls. The router maintains a

Redis cache keyed on task type + input hash. Cache hits bypass the LLM entirely, reducing both cost and latency to near-zero.

Task Type	Cache TTL	Rationale
Commodity classification	30 days	Tariff schedules change infrequently
Compliance screening	24 hours	Sanctions lists update daily
Entity resolution	7 days	Entity data is relatively stable
Document generation	Never cached	Each document is unique to its shipment
Carrier recommendations	1 hour	Rates and availability change frequently

Confidence Scoring

Every agent output includes a confidence score (0.0 to 1.0). The model router tracks confidence distributions per model per task type. If a model's average confidence for a task type drops below a configurable threshold, the router automatically switches to the fallback model and alerts the engineering team. This provides early warning of model degradation — a critical capability when relying on third-party LLM APIs that can change behavior without notice.

Prompt Version Management

Agent prompts are stored as versioned files in the repository (`shared/prompts/`). Each prompt file includes the system prompt, few-shot examples, output schema definition, and performance metadata. The model router loads the correct prompt version for each task. Prompt updates deploy through the normal CI/CD pipeline — a prompt change is a code change, with the same review, testing, and rollback capabilities. A/B testing of prompt versions is supported: the router splits traffic between two versions and compares accuracy, latency, and cost before committing.

Cost Optimization

Three strategies reduce LLM costs progressively over time:

Rules before models. The rules engine handles deterministic cases without invoking any LLM. As the Company Memory Graph accumulates more precedents, more cases become

deterministic. After 6 months, the rules engine is expected to handle 30-50% of classification and compliance tasks without LLM calls.

Cache before compute. The response cache eliminates repeat calls for identical inputs. A forwarder shipping similar commodities on similar routes sees cache hit rates of 40-60% for classification tasks within the first month.

Downgrade when proven. Tasks that consistently produce high-confidence results on GPT-5.4 can be downgraded to GPT-5 Mini. The model router tracks per-task confidence metrics and recommends downgrades when a cheaper model produces equivalent results. Downgrades are manual decisions reviewed by the engineering team — not automatic actions.

---

8. AWS Infrastructure Design

Design Principles

Services are independently deployable — a bug in document processing does not take down shipment creation. Communication is asynchronous by default — services talk through queues, not direct HTTP calls, so a slow agent does not block the operator. Every service is stateless — state lives in the database and cache, not in the container, so any container can be replaced without data loss.

Service Architecture

Service	Responsibility	Scaling Trigger	Resources
API Service	All HTTP requests from the frontend. Authentication, authorization, request validation, response formatting. REST API for web app and future integrations.	Request count (auto-scale at 500 req/sec)	1 vCPU, 2 GB
Agent Worker Service	Runs LangGraph orchestration. Consumes tasks from SQS, invokes LLM APIs via Model Router, produces structured outputs.	Queue depth (auto-scale at 10+ pending messages)	2 vCPU, 4 GB

Document Processing Service	OCR (AWS Textract), PDF generation, document template rendering, S3 storage.	Queue depth	2 vCPU, 4 GB
Email Ingestion Service	Receives inbound emails from SES, parses MIME, extracts attachments, publishes ingestion tasks to SQS.	Email volume (auto-scale at 100/hour)	1 vCPU, 2 GB
Workflow Orchestrator	Deterministic workflow engine and rules engine. Evaluates business rules, manages state transitions, approval gates. Separate from Agent Worker to ensure workflow logic is never blocked by LLM latency.	Active workflow count	1 vCPU, 2 GB
Model Routing Service	LLM API connections, model selection, fallback chains, caching, prompt management, cost tracking. All LLM calls go through this service.	LLM request volume	1 vCPU, 2 GB

AWS Service Map

AWS Service	Purpose
ECS / Fargate	Serverless containers for all six services. Blue-green deployment, per-service auto-scaling.
RDS PostgreSQL 16	System of record. Multi-AZ, db.r6g.large, automated backups, pgvector for embeddings.

S3	Document storage, raw email archive, frontend hosting. Versioned buckets, 7-year retention, AES-256 encryption.
SQS	Task queues per agent type: `intake-tasks`, `classification-tasks`, `compliance-tasks`, `document-tasks`, `billing-tasks`, `migration-tasks`. Dead-letter queues for failures.
SNS	Event broadcasting: `shipment-created`, `shipment-updated`, `compliance-alert`, `document-generated`, `invoice-created`.
SES	Inbound email at `*@ops.dynasties.ai` (triggers Lambda → SQS). Outbound for invoices, notifications, audit reports.
CloudFront	CDN for frontend. Edge locations globally, HTTPS only, custom domain `app.dynasties.ai`.
CloudWatch	Monitoring: service health, LLM costs per model, queue depths, API latency, error rates.
ECR	Docker container registry. Lifecycle policy retains last 10 images per service.
ElastiCache Redis	Session management, entity lookups, classification cache, rate limiting.
Secrets Manager	API keys (OpenAI, Anthropic, Google), database credentials, JWT secrets. Automatic rotation.
WAF	Rate limiting, SQL injection protection, geo-blocking.

Cost Estimation

Component	500 Shipments/Month	5,000 Shipments/Month
ECS Fargate (6 services)	$200-$400	$1,000-$2,000
RDS PostgreSQL (Multi-AZ)	$400	$400-$800
LLM API costs (blended)	$400-$800	$3,500-$7,000
S3 + CloudFront	$50	$200

SQS + SNS + SES	$20	$100
ElastiCache Redis	$100	$200
CloudWatch + WAF	$75	$200
Secrets Manager	$10	$10
Total	**$1,255-$1,855/month**	**$5,610-$10,510/month**

---

9. CI/CD Pipeline

Development Environment

All development happens in **Replit**. AI coding tools — Replit Agent, Claude, Cursor — generate and modify code in real-time. Engineers work in Replit workspaces connected to the GitHub repository. Every save pushes to a feature branch. Code reviews happen on GitHub via pull requests.

The local development environment runs the full stack using Docker Compose. A single `docker-compose up` starts all six services, a local PostgreSQL instance, a local Redis instance, and local SQS/SNS emulators (LocalStack). Code developed in Replit behaves identically to production.

Deployment Pipeline

Backend services:

Plain Text

```
Developer pushes code (Replit → GitHub feature branch)
→ Pull request opened
→ GitHub Actions: run test suite (unit + integration)
→ Code review and approval
→ Merge to main
→ GitHub Actions CI/CD:
    1. Run full test suite
    2. Build Docker images for modified services
    3. Push images to AWS ECR
    4. Update ECS task definitions
    5. Deploy to staging (ECS staging cluster)
    6. Run smoke tests against staging
    7. If pass: blue-green deploy to production
    8. Monitor health for 10 minutes
```

```
    9. If healthy: complete deployment
    10. If unhealthy: automatic rollback
```

Frontend:

```
Merge to main
→ GitHub Actions: build React application (npm run build)
→ Upload built assets to S3 (frontend bucket)
→ Invalidate CloudFront cache
→ Live globally within 60 seconds
```

Repository Structure

```
dynasties/
├── services/
│   ├── api/                    # API Service
│   ├── agent-worker/           # Agent Worker (LangGraph)
│   ├── document-processor/     # Document Processing
│   ├── email-ingestion/        # Email Ingestion
│   ├── workflow-orchestrator/  # Workflow Orchestrator
│   └── model-router/           # Model Routing
├── frontend/
│   └── client/                 # React + TypeScript + Tailwind
├── shared/
│   ├── schemas/                # Data schemas (JSON Schema)
│   ├── rules/                  # Business rules (YAML)
│   └── prompts/                # Versioned agent prompts
├── infrastructure/
│   ├── docker/                 # Dockerfiles per service
│   ├── docker-compose.yml      # Local dev environment
│   ├── terraform/              # AWS infrastructure as code
│   └── github-actions/         # CI/CD workflows
├── migrations/                 # Database migrations
└── tests/                      # Unit, integration, e2e
```

Environment Configuration

Environment	Purpose	Infrastructure
Local (Replit)	Development and debugging	Docker Compose + LocalStack

Staging	Pre-production validation	AWS ECS (single-AZ, smaller instances)
Production	Live operations	AWS ECS (Multi-AZ, auto-scaling)

Secrets are managed through AWS Secrets Manager and injected into containers at runtime. No secrets in code or environment files.

---

10. 30-Day Build and Launch Plan

Team and Assumptions

Two engineers (one full-stack, one AI/ML) and the founder. Development in Replit with AI coding assistance. AWS infrastructure provisioned via Terraform on Day 1. Pilot customer identified before Day 1. Insurance, advanced analytics, and deep carrier API integrations deferred to Weeks 5-8.

Week 1: Foundation (Days 1-7)

Day	Deliverable	Owner
1	AWS infrastructure via Terraform: ECS cluster, RDS PostgreSQL, S3 buckets, SQS queues, SNS topics, SES inbound, ECR, CloudFront, ElastiCache Redis. CI/CD pipeline in GitHub Actions.	Full-Stack
1-2	Core data model: Shipments, Parties, Commodities, Containers, Charges, Documents, Users, AgentState, MemoryGraph. PostgreSQL schema with pgvector.	Full-Stack
2-3	Email Ingestion Service: SES inbound → Lambda → S3 → SQS → service. MIME parsing, attachment extraction, document type identification.	AI/ML

3-4	Model Routing Service: multi-model connections (OpenAI, Anthropic, Google), routing logic, fallback chains, Redis caching, cost tracking.	AI/ML
4-5	Intake Agent: Claude Sonnet 4.6 for document parsing. Structured JSON output from emails and attachments. Field-level confidence scores.	AI/ML
5-6	Entity Management: CRUD for customers, carriers, vendors, agents. Fuzzy search, deduplication, auto-creation from ingested data. Entity Agent for resolution.	Full-Stack
6-7	Migration Studio backend: Magaya CSV/XML parser with XSD-derived field mappings. CargoWise common field mappings. AI-assisted mapping for unknown formats. Batch import with validation.	AI/ML
7	Workflow Orchestrator: LangGraph workflow definitions for "New Shipment from Email" and "Import from Legacy System." State management, checkpointing, approval gates.	AI/ML

Week 1 Milestone: Forward an email → see a structured shipment draft. Upload a Magaya CSV → see imported data with validation results. The core pipeline works end-to-end.

Week 2: Intelligence Layer (Days 8-14)

Day	Deliverable	Owner
8-9	Classification Agent: HS code suggestion with tariff	AI/ML

	schedule RAG. Confidence scoring. Precedent caching via Company Memory Graph.	
8-9	Compliance Agent: restricted party screening against SDN, Entity List, Denied Persons, UN/EU lists. Mandatory, non-bypassable.	AI/ML
10-11	Rules Engine: YAML-based business rules, evaluation pipeline, deterministic handling for known patterns. Initial rule set for common commodities and compliance.	AI/ML
10-11	Company Memory Graph: PostgreSQL + pgvector schema for storing entity relationships, commodity precedents, customer patterns, pricing behavior. Query API for agents.	AI/ML
12-13	Agentic Workbench frontend: React + Tailwind. Operational queues (Drafts, Missing Info, Compliance Flags, Approvals). Shipment detail view with confidence indicators. Keyboard shortcuts. Approval workflow.	Full-Stack
13-14	Document Agent: BOL, AWB, Cargo Manifest, Arrival Notice template engine. Claude Sonnet 4.6 for intelligent filling. PDF generation.	Full-Stack
14	End-to-end integration: email → intake → rules → classification + compliance (parallel) → shipment draft → Workbench → approval → document generation.	Both

Week 2 Milestone: Complete email-to-shipment-with-documents flow in the Agentic Workbench. Operator sees drafts with HS codes, compliance results, confidence scores, and generated documents. Approve with one click.

Week 3: Financial Loop and Natural Language (Days 15-21)

Day	Deliverable	Owner
15-16	Pricing Agent: charge type management, rate table CRUD, auto-calculation from shipment parameters, margin analysis, anomaly detection against historical patterns.	Full-Stack
16-17	Invoicing: invoice generation from approved charges, PDF output, email delivery via SES, accounts receivable tracking with aging.	Full-Stack
17-18	Natural Language Interface: GPT-5.4 intent parsing, routing to workflows, structured responses. "Create a shipment from Shanghai to LA, electronics, 2 TEU" → full draft.	AI/ML
18-19	Carrier Agent: carrier database, recommendation engine (cost + transit time + reliability ranking), historical performance from imported data, Company Memory Graph preferences.	AI/ML
19-20	Memory Agent: captures operator corrections, updates Company Memory Graph, generates new rules from repeated patterns, tracks learning metrics.	AI/ML
20-21	Migration Studio frontend: guided import wizard, file upload, field mapping review, validation summary,	Full-Stack

	enrichment results, import confirmation. Free Intelligence Audit report generation.	

Week 3 Milestone: Complete financial loop closed — shipment → documents → charges → invoice → email delivery. Natural language shipment creation works. Migration Studio handles Magaya imports with AI enrichment. Free Intelligence Audit produces PDF report from uploaded data.

Week 4: Security, Polish, and Launch (Days 22-30)

Day	Deliverable	Owner
22-23	Authentication: OAuth 2.0, JWT, role-based access (Admin, Manager, Operator, Viewer).	Full-Stack
23-24	Onboarding flow: account creation, email forwarding setup, guided first-shipment experience, Migration Studio walkthrough.	Full-Stack
24-25	Security: input validation, rate limiting, CORS, SQL injection prevention, XSS protection, encryption at rest (KMS) and in transit (TLS 1.3).	Full-Stack
25-26	Workbench polish: batch operations, filtering, sorting, keyboard navigation, responsive design, loading states, error handling, empty states.	Full-Stack
26-28	End-to-end testing with real shipment data from pilot customer. Bug fixes, performance optimization, edge case handling.	Both
28-29	Production deployment: Terraform apply, DNS	Full-Stack

	(`app.dynasties.ai`), SSL, CloudWatch alarms, error tracking. Staging validation.	
29-30	Launch with pilot customer. White-glove onboarding: founder walks customer through account setup, email forwarding, data import, and first live shipment.	Founder

Day 30 Milestone: Dynasties is live. First pilot customer is processing real shipments through the Agentic Workbench. The Company Memory Graph is learning. The system handles the complete operational loop from email ingestion through invoicing. Monitoring and alerting are active.

Post-Launch: Weeks 5-8

Week	Priority	Rationale
5-6	Embedded insurance quoting (Breeze AI or Loadsure API)	Revenue layer — commission on every insured shipment
5-6	QuickBooks / Xero invoice sync	Eliminates manual accounting entry
5-7	Carrier tracking API integrations (Maersk, MSC, CMA CGM)	Automates milestone updates, enables ETA predictions
6-8	Customer-facing tracking portal	Differentiation, reduces "where is my shipment" calls
6-8	Advanced analytics dashboard	Retention driver, strategic advisor positioning
7-8	Second and third pilot customers	Validates platform across different operational patterns

---

Learning Loops

Every operator correction improves the system. This is not a vague aspiration — it is an

engineered feedback mechanism with specific data flows.

When an operator modifies an agent's output, the correction event is captured with full context: the original agent recommendation, the operator's modification, the shipment context, and the operator's scope selection (this shipment only, this customer, or global). The Memory Agent processes the correction and determines the appropriate response.

Correction Type	System Response	Impact Timeline
HS code changed	Stores precedent in Company Memory Graph. If same correction occurs 3+ times, promotes to deterministic rule in Rules Engine.	Immediate for same commodity; rule after 3 occurrences
Carrier changed	Records preference in customer profile. Adjusts Carrier Agent weighting for this customer-lane combination.	Next shipment on same lane
Charge adjusted	Updates pricing pattern in Memory Graph. Flags future charges that deviate from the corrected pattern.	Next similar shipment
Entity corrected	Updates entity record. Improves Entity Agent's fuzzy matching for similar names/addresses.	Immediate
Document modified	Records template preference. Document Agent uses corrected format for future documents of same type for this customer.	Next document generation
Compliance override (with justification)	Records exception with regulatory justification. Does not create a rule — compliance exceptions are always case-specific.	No automatic propagation

Over time, the ratio of agent-generated outputs that operators approve without modification increases. This metric — the **auto-approval rate** — is the single most important KPI for the platform. A new customer on Day 1 might have a 60% auto-approval

rate. After 3 months, it should be 85%. After 12 months, 95%+. The remaining 5% represents genuinely novel situations that require human judgment — and those situations train the system for the next novel case.

References

[1] What Is Blue Ocean Strategy? — Blue Ocean Strategy
[2] Zero to One — Peter Thiel
[3] Transportation Management System Market Size — Grand View Research
[4] WiseTech Global Investor Relations
[5] Descartes Systems Group Investor Relations
[6] e2open Investor Relations
[7] Best Multi-Agent Frameworks in 2026 — GurusUp
[8] Magaya Supply Chain — Working with Lists
[9] Introducing Claude Sonnet 4.6 — Anthropic
[10] OpenAI GPT-5 Model Guide — NxCode
[11] Gemini 2.5 Pro Specs and Pricing — GetDeploying